



Software Architecture Laboratory

Laboratory No. 8

Infrastructure as Code with Terraform on Azure

Public Load Balancer / Linux VMs / Remote State / GitHub Actions OIDC

Students:

Andersson David Sánchez Méndez

Cristian Santiago Pedraza Rodríguez

Elizabeth Correa Suárez

Juan Sebastian Ortega Muñoz

Instructor: Professor Javier Ivan Toquica Barrera

Course: Software Architecture (ARSW)

Institution: Escuela Colombiana de Ingeniería Julio Garavito

May 2026

0 Contents

1 Objective	3
2 Target Architecture	3
2.1 Component diagram	3
2.2 Sequence diagram — end-user request	3
2.3 Sequence diagram — CI/CD with OIDC	4
3 Part 1 — Modular Terraform Code	4
3.1 Variable validation	4
3.2 Cloud-init bootstrap	4
4 Part 2 — Load Balancer & NSG Module	4
5 Part 3 — Remote State & Locking	5
6 Part 4 — GitHub Actions with OIDC	5
7 Part 5 — Operational Validation	6
8 Security Posture	6
9 Cost Analysis	6
10 Optional Challenges Implemented	6
10.1 Challenge #1 — Azure Bastion (no public SSH)	6
10.2 Challenge #2 — Cost Management budget with email alerts	7
11 Video Evidence	7
12 Conclusions	7
12.1 Lessons Learned	7
13 References	8

1 Objective

This laboratory modernises the classic Azure load-balancing exercise by expressing the entire infrastructure as code with **Terraform**. Building on the cloud foundations established in previous laboratories, the team designed and provisioned a reproducible deployment that exposes two Linux web servers behind a public, Layer-4 **Azure Load Balancer**, backs the configuration with a remote, lock-protected Terraform **state**, and integrates a **GitHub Actions** pipeline that authenticates against Azure using **OpenID Connect** federation instead of long-lived secrets.

Lab #8 Objectives

1. Model an Azure deployment with Terraform: providers, modules, variables, outputs and remote state.
2. Provision a high-availability workload behind a Standard SKU Load Balancer with at least two Linux backends.
3. Apply a minimum security baseline: NSG with explicit allow-list, SSH key-only authentication, naming conventions and tags.
4. Externalise state to an Azure Storage Account using blob lease locking to prevent concurrent applies.
5. Build a CI/CD pipeline in GitHub Actions that performs **plan** on every pull request and gated **apply** on manual dispatch, all under OIDC.
6. Validate that the LB round-robins traffic across the backends, observe approximate costs and destroy the deployment safely.

2 Target Architecture

The deployment is partitioned into three Terraform modules so that each concern is independently versionable and reviewable. A single Resource Group acts as the container, which keeps the cleanup operation atomic.

Module	Responsibility
vnet	Virtual Network and the web / mgmt subnets.
compute	NICs and Linux VMs bootstrapped by cloud-init.
lb	Public Load Balancer, health probe, rule and NSG

Key Design Decision

The choice of **Layer-4 Load Balancer** over Application Gateway (L7) is deliberate. The workload serves identical static HTML over plain HTTP, with no path routing, no TLS termination and no WAF requirements. A Standard LB delivers sub-millisecond latency and one-tenth of the hourly cost of Application Gateway. Should the workload need TLS or path-based routing in the future, only the **lb** module would be swapped, leaving **vnet** and **compute** untouched. This is the precise benefit modular Terraform was designed to deliver.

2.1 Component diagram

The component diagram (Figure 1) is rendered with **advanced Mermaid notation** (subgraphs, classed nodes, custom theming) and captures the four logical layers of the deployment: identity (Azure AD + OIDC), remote state (Storage Account + container + blob lease), workload (Resource Group containing VNet, NSG, NICs, VMs and the LB) and the external actors (developer CLI, GitHub runner and the end user). The full Mermaid source lives in `docs/DIAGRAMS.md`.

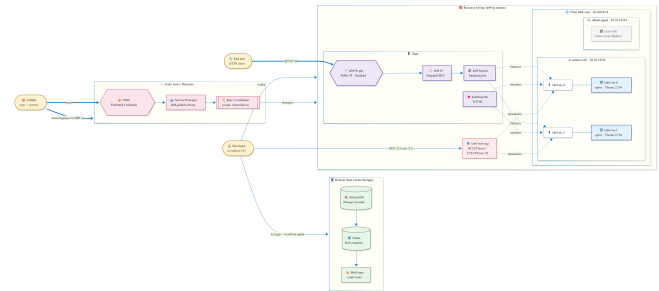


Figure 1: Component diagram of the Lab #8 deployment: identity federation, remote state backend, edge layer and load-balanced VMs.

2.2 Sequence diagram — end-user request

Figure 2 depicts what happens when an end user issues `curl http://<lb_public_ip>`: name resolution, TCP handshake intercepted by the LB, 5-tuple hashing that selects a backend NIC, NSG acceptance of the inbound 80/TCP flow, and the response returning back through the same path. The diagram also visualises the periodic TCP/80 health probes that keep the backend pool membership accurate.

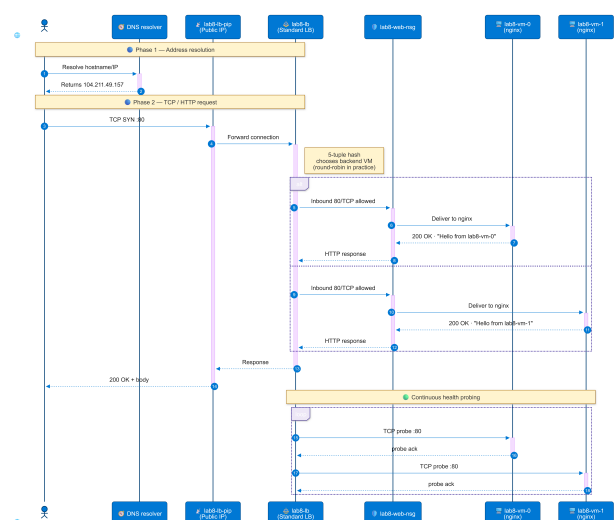


Figure 2: Request flow through the Public IP, the Load Balancer 5-tuple hash and the NSG before the nginx VM responds with the page that embeds its hostname.

2.3 Sequence diagram — CI/CD with OIDC

Figure 3 complements the previous one by showing the operational path: an engineer triggers a workflow, GitHub mints a short-lived OIDC JWT, Azure AD swaps it for an access token, the runner acquires a blob lease on the state file, runs Terraform against ARM and finally posts the plan back to the PR.

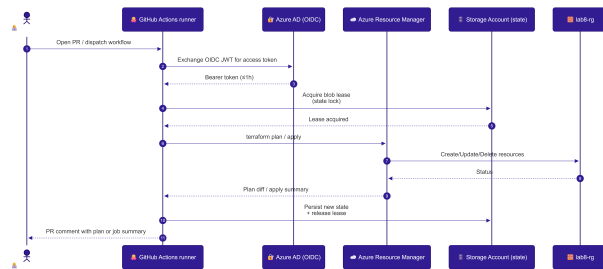


Figure 3: CI/CD sequence: GitHub Actions trades the OIDC JWT for an Azure AD token, locks the remote state and runs Terraform.

3 Part 1 — Modular Terraform Code

The root composition wires the three modules together while keeping provider configuration centralised in `providers.tf`. The `azurerm` backend block is intentionally empty, so the actual storage configuration is injected at `terraform init` time via `-backend-config=backend.hcl`, keeping subscription identifiers out of source control.

```
1 resource "azurerm_resource_group" "rg" {
2   name     = "${var.prefix}-rg"
3   location = var.location
4   tags     = var.tags
5 }
6
7 module "vnet" {
8   source                = "../modules/vnet"
9   resource_group_name   = azurerm_resource_group.rg.name
10  location               = azurerm_resource_group.rg.location
11  prefix                 = var.prefix
12  tags                   = var.tags
13 }
14
15 module "compute" {
16   source                = "../modules/compute"
17   resource_group_name   = azurerm_resource_group.rg.name
18   location              = azurerm_resource_group.rg.location
19   prefix                 = var.prefix
20   admin_username        = var.admin_username
21   ssh_public_key         = file(var.ssh_public_key)
22   subnet_id             = module.vnet.subnet_web_id
23   vm_count               = var.vm_count
24   cloud_init             = file("${path.module}/cloud-init-init.yaml")
25   tags                   = var.tags
26 }
27
28 module "lb" {
29   source                = "../modules/lb"
30   resource_group_name   = azurerm_resource_group.rg.name
31   location              = azurerm_resource_group.rg.location
32   prefix                 = var.prefix
33   backend_nic_ids       = module.compute.nic_ids
34   allow_ssh_from_cidr   = var.allow_ssh_from_cidr
35   tags                   = var.tags
36 }
```

Listing 1: Root composition (`infra/main.tf`) wiring the three modules

3.1 Variable validation

Every root variable is typed and validated where it makes sense, so the plan fails fast on bad inputs instead of producing partial deployments. For example, `vm_count` is bounded between 2 and 5 to enforce high availability without letting the lab spend run away.

```
1 variable "prefix" {
2   type        = string
3   description = "Short identifier prepended to every resource name."
4   validation {
5     condition = can(regex("^[a-z][a-z0-9-]{1,10}$", var.prefix))
6     error_message = "prefix must be 2-11 lowercase alphanumerics."
7   }
8 }
9
10 variable "vm_count" {
11   type        = number
12   description = "Number of Linux VMs registered behind the LB."
13   validation {
14     condition = var.vm_count >= 2 && var.vm_count <= 5
15     error_message = "vm_count must be between 2 and 5."
16   }
17 }
```

Listing 2: Validated input variables (excerpt of `infra/variables.tf`)

3.2 Cloud-init bootstrap

The Linux VMs install `nginx` on first boot and write a tiny banner that embeds the hostname. This makes the LB round-robin behaviour observable end-to-end with a simple `curl`.

```
#cloud-config
package_update: true
packages:
- nginx
runcmd:
- echo "<h1>Hello from $(hostname)</h1>" > /var/www/html/index.nginx-debian.html
- systemctl enable nginx
- systemctl restart nginx
```

Listing 3: `infra/cloud-init.yaml`: `nginx` installation with hostname banner

4 Part 2 — Load Balancer & NSG Module

The `lb` module is where the design tradeoffs concentrate. It provisions a Standard SKU Public IP, a Standard SKU Load Balancer with a single public frontend, the backend address pool, the TCP/80 health probe, the load balancing rule and the NSG. The NSG is bound at NIC level (rather than subnet level) so subsequent tiers can declare their own posture without inheritance surprises.

```
1 resource "azurerm_lb_probe" "probe" {
2   name     = "http-80"
3   loadbalancer_id = azurerm_lb.lb.id
4   protocol  = "Tcp"
5   port      = 80
6 }
```

```

6 }
7
8 resource "azurerm_lb_rule" "rule" {
9   name                        = "http-80"
10  loadbalancer_id             = azurerm_lb.lb.
11    id
12  protocol                    = "Tcp"
13  frontend_port               = 80
14  backend_port                = 80
15  frontend_ip_configuration_name = "Public"
16  backend_address_pool_ids     = [
17    azurerm_lb_backend_address_pool.bepool.id]
18  probe_id                    =
19    azurerm_lb_probe.probe.id
20 }
21
22 resource "azurerm_network_security_group" "nsg" {
23   name                        = "${var.prefix}-web-nsg"
24   location                    = var.location
25   resource_group_name         = var.resource_group_name
26
27   security_rule {
28     name                        = "Allow-HTTP-
29       Internet"
30     priority                    = 100
31     direction                  = "Inbound"
32     access                     = "Allow"
33     protocol                   = "Tcp"
34     destination_port_range     = "80"
35     source_address_prefix      = "*"
36     source_port_range          = "*"
37     destination_address_prefix = "*"
38   }
39
40   security_rule {
41     name                        = "Allow-SSH-From-
42       Operator"
43     priority                    = 110
44     direction                  = "Inbound"
45     access                     = "Allow"
46     protocol                   = "Tcp"
47     destination_port_range     = "22"
48     source_address_prefix      = var.
49       allow_ssh_from_cidr
50     source_port_range          = "*"
51     destination_address_prefix = "*"
52   }
53 }

```

Listing 4: Standard LB, probe, rule and NSG (excerpt of modules/lb/main.tf)

Note

The Standard SKU was selected over Basic because Public IPs of Standard SKU are mandatory for the Standard LB family, and Basic SKU is on the deprecation track in Azure since September 2025. Choosing Standard preserves availability zone readiness without committing to it on day one.

5 Part 3 — Remote State & Locking

Local Terraform state is incompatible with collaborative work: any parallel `apply` would race and corrupt the file. The team provisioned a dedicated Storage Account (`sttfstate658` in resource group `rg-tfstate-lab8`) with a single private container (`tfstate`). Terraform automatically acquires a blob lease on the state file before each operation, which guarantees mutual exclusion and survives crashed runs (the lease auto- expires after one minute).

```

SUFFIX=$RANDOM
LOCATION=eastus
RG=rg-tfstate-lab8
STO=sttfstate${SUFFIX}
CONTAINER=tfstate
az group create -n $RG -l $LOCATION

```

```

az storage account create -g $RG -n $STO -l
  $LOCATION \
  --sku Standard_LRS --encryption-services blob
az storage container create --name $CONTAINER --
  account-name $STO

```

Listing 5: Bootstrap of the remote state backend (run once)

The corresponding `backend.hcl` (kept out of source control) is the only file containing storage names; `providers.tf` declares an empty backend `"azurerm"` block that the CLI hydrates at init time.

6 Part 4 — GitHub Actions with OIDC

Storing an Azure Service Principal client secret inside GitHub **Secrets** works, but rotates poorly and exposes a long-lived credential on every job log leak. Instead, the team configured **OpenID Connect federation** between GitHub Actions and Azure AD. Each workflow run mints a short-lived OIDC JWT, swaps it for an Azure AD access token via `azure/login@v2`, and uses that ephemeral token to call ARM and the state Storage Account. The federation trust itself lives entirely on the Azure side as an *App Registration federated credential* bound to the GitHub repository's OIDC subject.

```

permissions:
  id-token: write      # required for OIDC token
  request
  contents: read
  pull-requests: write

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with: { terraform_version: "1.9.5" }
      - run: terraform fmt -check -recursive
      - uses: azure/login@v2
        with:
          client-id:      ${ secrets.
            AZURE_CLIENT_ID }
          tenant-id:      ${ secrets.
            AZURE_TENANT_ID }
          subscription-id: ${ secrets.
            AZURE_SUBSCRIPTION_ID }
      - run: terraform init -backend=false
      - run: terraform validate -no-color
      - run: terraform validate -no-color
        working-directory: infra

plan:
  needs: lint
  if: github.event_name == 'pull_request'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: hashicorp/setup-terraform@v3
      with: { terraform_version: "1.9.5" }
    - uses: azure/login@v2
      with:
        client-id:      ${ secrets.
          AZURE_CLIENT_ID }
        tenant-id:      ${ secrets.
          AZURE_TENANT_ID }
        subscription-id: ${ secrets.
          AZURE_SUBSCRIPTION_ID }
    - run: terraform init \
      -backend-config="resource_group_name=${ secrets.BACKEND_RG }" \
      -backend-config="storage_account_name=${ secrets.BACKEND_SA }" \
      -backend-config="container_name=${ secrets.BACKEND_CONTAINER }" \
      -backend-config="key=${ secrets.

```

```

    BACKEND_KEY }}"
    terraform plan -var-file=env/dev.tfvars -
    no-color -out=tfplan

```

Listing 6: Lint + Plan jobs (excerpt of `.github/workflows/terraform.yml`)

Pipeline behaviour

- Every pull request to `main` runs `fmt`, `validate` and `plan`; the plan diff is posted back to the PR as a sticky comment so reviewers see the impact without having to fetch the branch locally.
- The `apply` and `destroy` jobs are gated behind the `production` GitHub Environment, which requires explicit reviewer approval and restricts deployments to the `main` branch.
- No long-lived secret leaves Azure: the `azure/login@v2` step exchanges the GitHub OIDC JWT for a 1-hour AAD token.

7 Part 5 — Operational Validation

After a successful `terraform apply`, the team verified the deployment end-to-end. Outputs of the recorded run:

```

$ terraform output
lb_public_ip      = "104.211.49.157"
resource_group_name = "lab8-rg"
vm_names         = [
  "lab8-vm-0",
  "lab8-vm-1",
]

```

Listing 7: Outputs surfaced by the root module after `apply`

A small loop hitting the Load Balancer six times demonstrates that successive requests rotate between both backends, confirming the backend pool is healthy and the 5-tuple hash distributes flows as expected.

```

$ for ($i=0; $i -lt 6; $i++) { curl http
  ://104.211.49.157 }
<h1>Hello from lab8-vm-0</h1>
<h1>Hello from lab8-vm-1</h1>
<h1>Hello from lab8-vm-0</h1>
<h1>Hello from lab8-vm-1</h1>
<h1>Hello from lab8-vm-0</h1>
<h1>Hello from lab8-vm-1</h1>

```

Listing 8: Round-robin verification against the LB Public IP

8 Security Posture

Even at lab scale, the team applied a baseline that mirrors what would be required in production:

- **NSG with explicit allow-list.** Only 80/TCP from the public Internet (required for the LB) and 22/TCP from the operator's /32 are permitted; everything else is denied by default.
- **SSH key-only authentication.** Password authentication is disabled by the `azurerm` provider when `admin_ssh_key` is supplied, structurally preventing password spray attacks.
- **Tagged ephemeral resources.** Every resource

carries `owner`, `course`, `env` and `expires` tags, making cost ownership and cleanup auditable.

- **No long-lived secret in CI/CD.** The OIDC federation means there is no Service Principal client secret to rotate or leak.
- **NIC-level NSG attachment.** Avoids subnet-level inheritance pitfalls when adding new tiers.

Key Design Decision

Exposing 22/TCP to the Internet, even restricted to a single /32, is still an attack surface. For production, the lab leaves `subnet-mgmt` pre-provisioned so **Azure Bastion** can be introduced later, removing public SSH entirely.

9 Cost Analysis

Approximate East US list prices, validated against the Azure pricing calculator on the deployment date:

Resource	Hourly	Daily	Monthly
2 × VM Standard_B1s	\$0.0104	\$0.50	\$15.20
Standard Load Balancer	\$0.0250	\$0.60	\$18.30
Standard Public IP (static)	\$0.0050	\$0.12	\$3.70
Storage Account (state, LRS)	\$0.0002	\$0.005	\$0.15
Total	\$0.041	\$1.23	\$37.35

An eight-hour development session therefore costs roughly USD \$0.33, which fits comfortably inside the USD \$100 Azure for Students credit and allows dozens of full lab cycles before the credit is exhausted.

10 Optional Challenges Implemented

The lab guide lists five optional challenges (*Retos*) and asks students to pick two or more. The team implemented two of them as **opt-in modules behind feature flags**, so the cheap base lab remains untouched and the additional resources only spin up when the team explicitly enables them. The full enable/disable procedure is documented in `docs/CHALLENGES.md`.

10.1 Challenge #1 — Azure Bastion (no public SSH)

The first challenge eliminates the public 22/TCP attack surface on the workload VMs. When `enable_bastion = true`, the `vnet` module provisions an additional subnet named exactly `AzureBastionSubnet` (10.10.3.0/26, both name and minimum size mandated by Azure), and a new `bastion` module deploys a Standard SKU Public IP plus the `azurerm_bastion_host` itself. Operators reach the VMs by clicking *Connect* → *Bastion* in the Azure portal, which brokers SSH/RDP over TLS without ever exposing port 22 to the Internet.

```

1 module "bastion" {
2   count      = var.enable_bastion ? 1 : 0
3   source     = "../modules/bastion"
4   resource_group_name = azurerm_resource_group.rg.name
5   location   = azurerm_resource_group.rg.location
6   prefix     = var.prefix

```

```

7   subnet_id      = module.vnet.
      subnet_bastion_id
8   sku            = var.bastion_sku
9   tags           = var.tags
10 }

```

Listing 9: Bastion module wired through a feature flag (`infra/main.tf`)

Key Design Decision

Azure Bastion is the most expensive single resource in this lab (approximately USD \$0.19/hour for the Basic SKU). The team kept the flag disabled by default so a forgotten `terraform apply` cannot silently drain the student credit; production deployments would also remove the `Allow-SSH-From-Operator` NSG rule entirely once Bastion is in place.

10.2 Challenge #2 — Cost Management budget with email alerts

The second challenge installs a hard *guardrail against runaway spend*: a Cost Management budget scoped to the lab Resource Group with three notification thresholds (50%, 90% and 100% of the configured monthly cap), all evaluated on *Actual* cost rather than *Forecast* to avoid noisy notifications during the first few days of the month. Budgets themselves are free; the only added cost is the email pipeline (also free).

```

1 module "budget" {
2   count      = var.enable_budget ? 1 : 0
3   source     = "../modules/budget"
4   prefix     = var.prefix
5   resource_group_id = azurerm_resource_group.rg.id
6   amount_usd = var.budget_amount_usd
7   contact_emails = var.budget_contact_emails
8   start_date  = var.budget_start_date
9 }
10
11 # inside modules/budget/main.tf:
12 notification {
13   enabled      = true
14   threshold    = 50
15   operator     = "GreaterThan"
16   threshold_type = "Actual"
17   contact_emails = var.contact_emails
18 }

```

Listing 10: Budget module: three notifications, gated by feature flag

Why these two challenges?

- **Bastion** attacks the largest residual security risk in the base architecture (the public SSH surface) without introducing a new technology stack to the team.
- **Budget alert** adds operational maturity at zero marginal cost — the perfect ergonomics win for a lab whose primary financial constraint is a fixed student credit.
- Both are wired with `count = var.enable_X ? 1 : 0`, which keeps the `terraform plan` for the base lab byte-identical to the pre-challenge version when the flags are off.

11 Video Evidence

A full demo recording captures the end-to-end execution: backend bootstrap, `terraform init` against the remote backend, `terraform apply`, curl loop verifying round-robin and `terraform destroy` for the cleanup. The video is hosted on Google Drive and linked from the repository README:

Demo recording

URL: <https://drive.google.com/file/d/1CAY1gmPZbe96VobD-xGRgOq5HYMfCj0u/view>

The recording showcases:

- Storage Account / container provisioning for remote state.
- Terraform initialising the AzureRM backend and downloading provider `azurerm v4.72.0`.
- Successful `plan` and `apply` producing the Resource Group, VNet, NICs, VMs, LB and NSG.
- Live curl loop alternating between `lab8-vm-0` and `lab8-vm-1`.
- Clean teardown via `terraform destroy` confirming a zero-leftover state.

12 Conclusions

Key Findings

1. **IaC dramatically reduces operational risk.** Replacing portal click-ops with declarative Terraform turned an error-prone, multi-step procedure into a single `apply`, fully reproducible and reviewable as code. Destroying the deployment is just as deterministic, which is critical for cost control on student credits.
2. **Module decomposition pays off immediately.** Splitting the workload into `vnet`, `compute` and `lb` kept each diff small and surfaced the precise seam at which a future Application Gateway upgrade would land.
3. **OIDC is the right default for cloud CI/CD.** Removing the long-lived Service Principal secret eliminated an entire class of credential-management problems. Federated credentials scoped per branch and per environment make the trust boundary easy to reason about.
4. **Standard L4 LB is the right tool for this workload.** Application Gateway would have multiplied cost and operational surface for routing features we do not consume. The clean module boundary keeps the upgrade path open.

12.1 Lessons Learned

- Pinning the AzureRM provider via `.terraform.lock.hcl` is non-negotiable: a minor provider bump silently changed defaults during early experimentation and caused unexpected plan diffs.

- NIC-level NSG attachment is more verbose than subnet-level, but completely avoids the NSG inheritance debugging that wasted time during earlier exercises.
- `terraform fmt -recursive` should be a pre-commit hook from day one; the CI `fmt -check` block in the Actions workflow catches drift but the developer feedback loop is much faster locally.
- Validating variables with `validation` blocks turns the `plan` step into a spec test: bad input fails before a single Azure API call is issued, which is exactly the feedback loop we want.

13 References

1. HashiCorp. *Terraform Language Documentation*. <https://developer.hashicorp.com/terraform/language>
2. Microsoft Azure. *Azure Load Balancer overview*. <https://learn.microsoft.com/en-us/azure/load-balancer/load-balancer-overview>
3. Microsoft Azure. *Use Azure Storage as a Terraform backend*. <https://learn.microsoft.com/en-us/azure/developer/terraform/store-state-in-azure-storage>
4. Microsoft Azure. *Configure a federated identity credential on an app for GitHub Actions*. <https://learn.microsoft.com/en-us/azure/developer/github/connect-from-azure-openid-connect>
5. HashiCorp. *AzureRM Provider Documentation (v4)*. <https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs>
6. Canonical. *cloud-init Documentation*. <https://cloudinit.readthedocs.io/en/latest/>
7. GitHub Docs. *About security hardening with OpenID Connect*. <https://docs.github.com/en/actions/deployment/security-hardening-your-deployments/about-security-hardening-with-openid-connect>